

DuocUC® INFORMÁTICA Y TELECOMUNICACIONES

Prevenir Overfitting

TLY1102

FORMAMOS PERONAS PARA UNA SOCIEDAD MEJOR



01

Identificar Overfitting

• Objetivo del entrenamiento

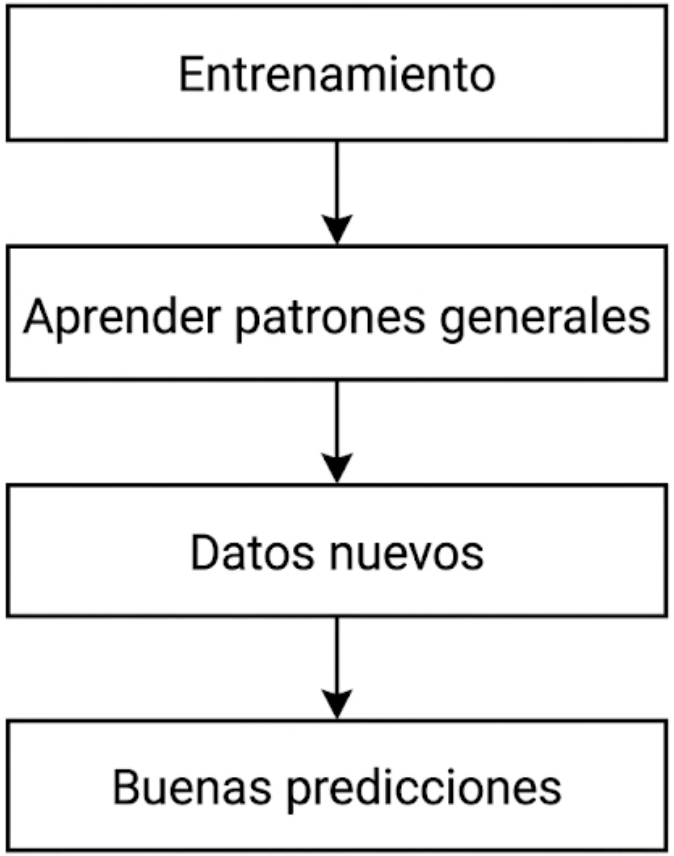
¿Qué significa que un modelo "aprenda"?

¿Cuál es el objetivo de entrenar una red neuronal?

El objetivo **no es tan solo obtener el mejor rendimiento** sobre los datos de entrenamiento (Optimización).

Sino **Generalizar y encontrar un balance con la Optimización.**

Es decir, **la capacidad de un modelo para extraer patrones subyacentes de los datos de entrenamiento y aplicarlos con precisión a datos nuevos, desconocidos y del mundo real**. Es la diferencia entre simplemente "memorizar" respuestas y realmente aprender, pero buscando un buen rendimiento.



• Memorizar no es aprender

Supongamos que entrenamos un modelo para reconocer gatos.

Todas las imágenes de entrenamiento contienen gatos naranjos.

El modelo aprende: Objeto Naranja = GATO



Luego aparece un zorro y el modelo responde: "Gato“, ¿Por qué?

Porque no aprendió qué es un gato. Aprendió una coincidencia.



Overfitting = memorizar características accidentales del conjunto de entrenamiento.

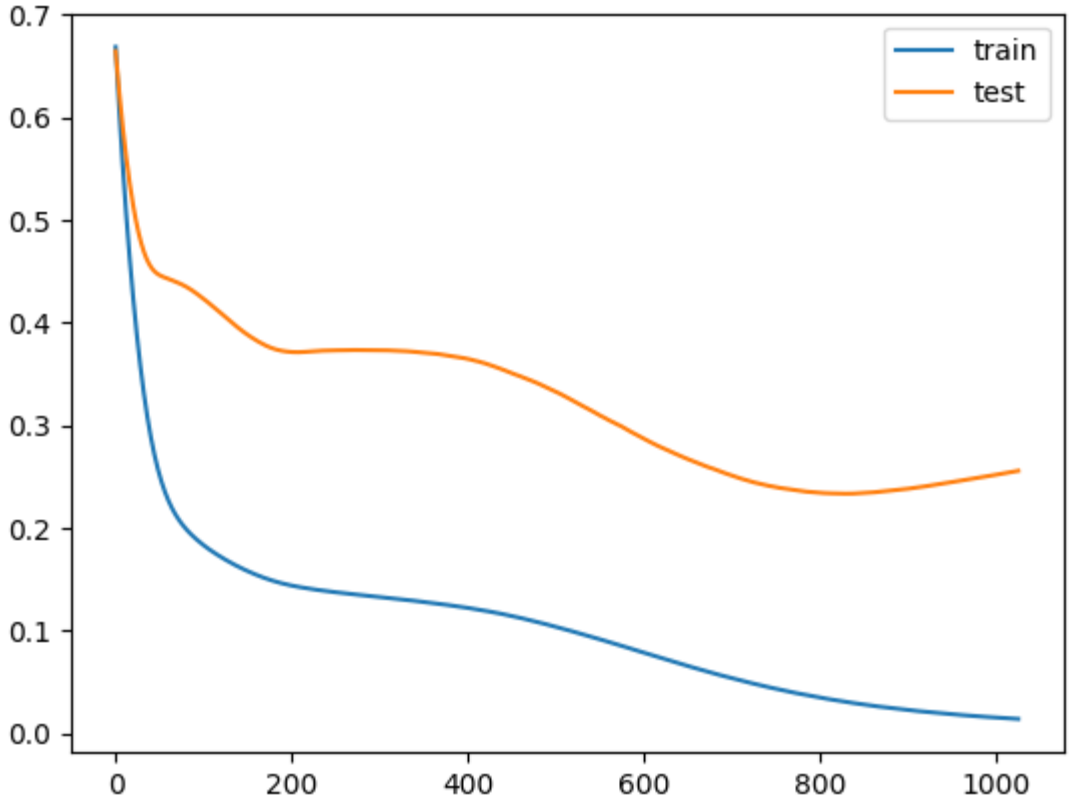
• ¿Cómo sabemos que un modelo está memorizando?

No podemos abrir la red neuronal y preguntarle qué aprendió. Entonces... Necesitamos evidencias.

La herramienta más simple son las curvas de aprendizaje.

Conjunto de entrenamiento (Training Set)
Es el conjunto de datos que la red neuronal utiliza para aprender.
El conjunto de entrenamiento sirve para enseñar al modelo.

Conjunto de validación (Validation Set)
El conjunto de validación contiene ejemplos que el modelo **nunca utiliza para actualizar sus pesos**.
Su única función es evaluar cómo se comporta el modelo frente a datos nuevos.
El conjunto de validación mide la capacidad de generalización del modelo.



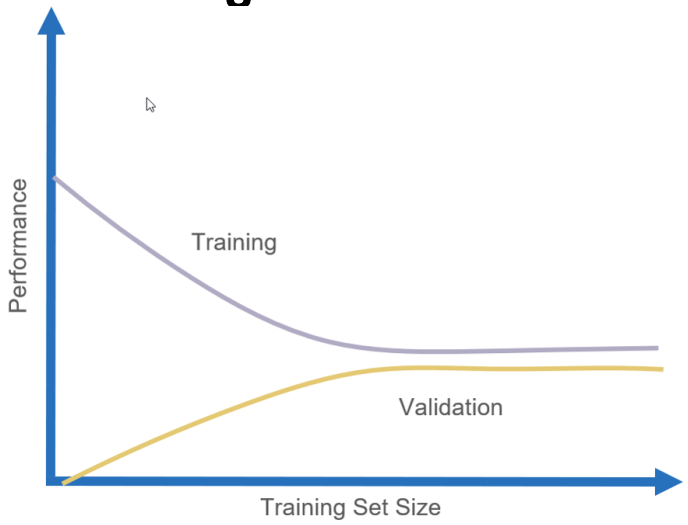
• Aprendiendo de las curvas

Curvas de aprendizaje

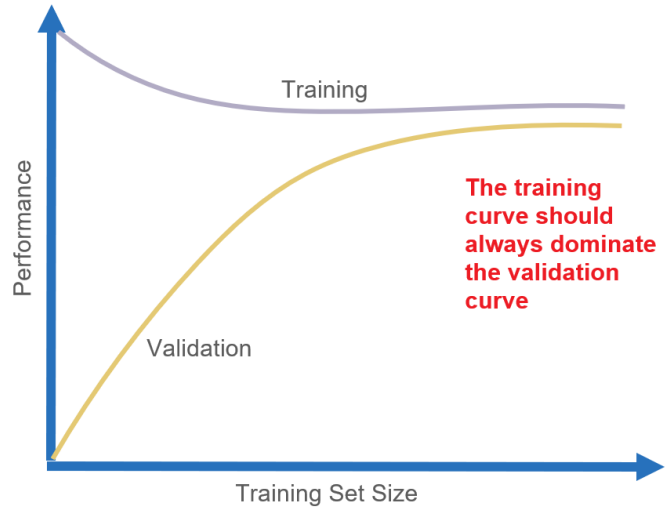
Las curvas de aprendizaje permiten visualizar el rendimiento de los datos de entrenamiento y validación (eje y) con respecto al tamaño de conjunto de datos de entrenamiento (eje x).

Responden a la pregunta:

"¿Aumentar la cantidad de datos ayudará a mejorar el rendimiento del modelo?".



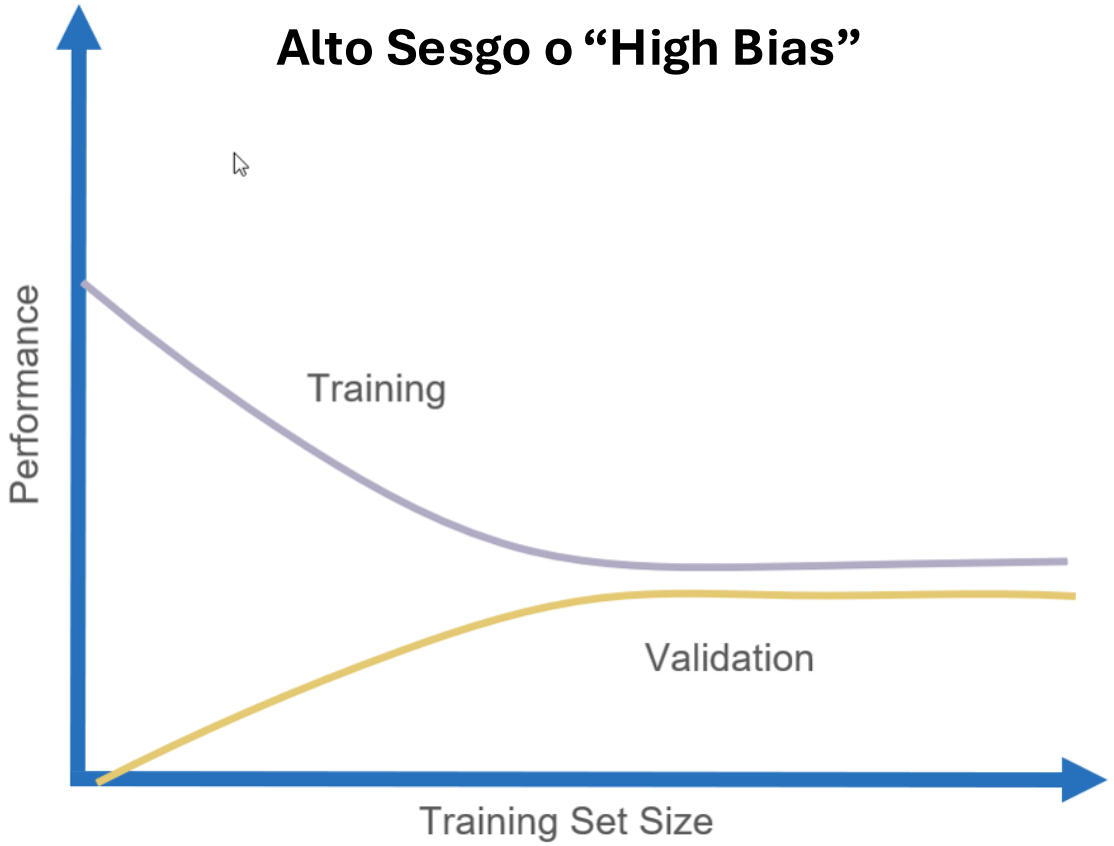




¿Cuál puede ser la curva ideal?

• Aprendiendo de las curvas

Underfitting



Aquí se observa una curva con un alto sesgo.

- El rendimiento del conjunto de validación y el de entrenamiento se mantienen estables, y a **medida que los datos aumentan, el rendimiento no varía**.

- El **rendimiento es bajo**. Esto indica un alto sesgo o una **falta de complejidad en el modelo**. El modelo **no encuentra el patrón en los datos**, y **añadir más datos no ayuda**.

Es decir,

- **Predicciones incorrectas:** El modelo no logra aprender la relación entre las variables de entrada y la salida esperada.

- **Baja complejidad:** Suele pasar por modelos lineales simples intentando resolver problemas altamente complejos.

- **Generaliza mal:** Aunque sea simple, no logra predecir correctamente cuando se enfrenta a datos que no ha visto antes.

• Aprendiendo de las curvas

Overfitting



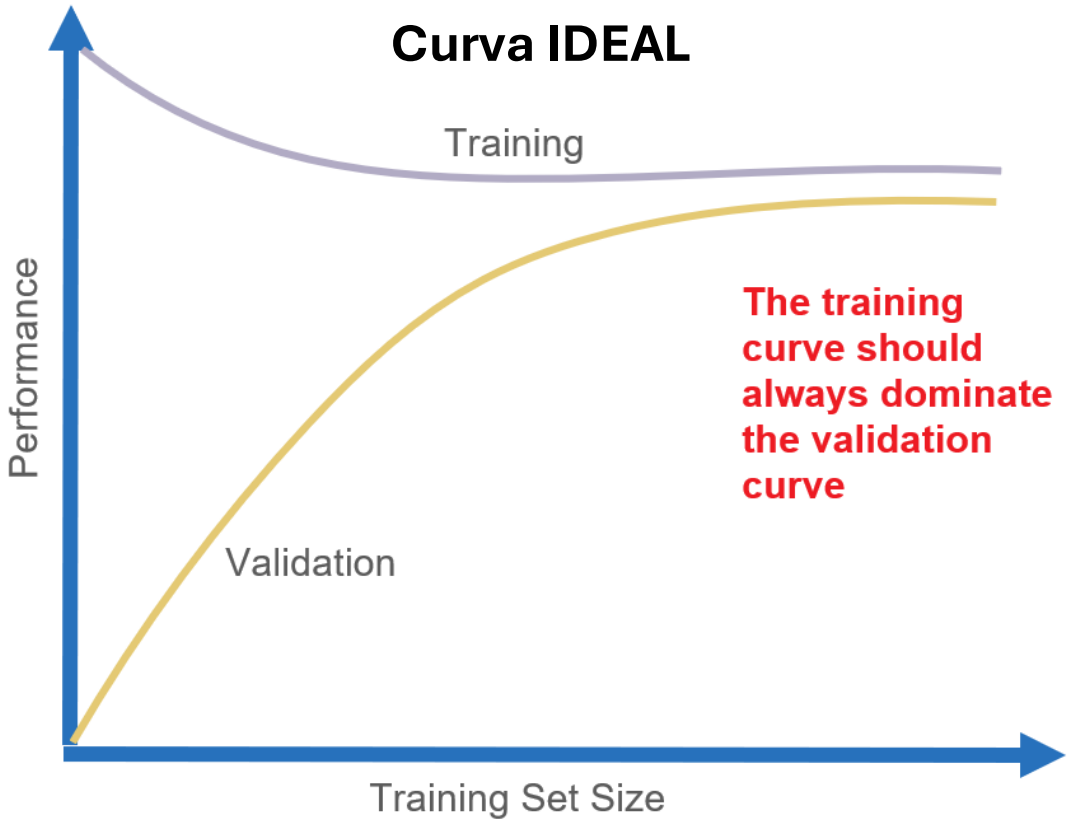
Aquí se observa un caso de sobreajuste. Fíjese en la diferencia entre los datos de entrenamiento y validación: **si el modelo funciona mucho mejor con los datos de entrenamiento, pero peor con los de validación, es probable que se trate de sobreajuste**. En este caso, no es necesario que el rendimiento del entrenamiento sea bueno o malo; **la diferencia es crucial**. Se puede producir **sobreajuste incluso con un rendimiento relativamente bajo o bueno**.

Lo fundamental es que la diferencia entre los datos de entrenamiento y validación no sea demasiado grande.

Además, **tenga en cuenta que obtener más datos no será de gran ayuda**; puede observar que los datos de validación se están estancando. **Si la tendencia fuera ascendente, cabría esperar que, con más datos, también mejorara**.

• Aprendiendo de las curvas

Curva IDEAL



Aquí se muestra una versión idealizada. **El conjunto de validación está aumentando y alcanza un alto nivel de rendimiento con respecto al conjunto de entrenamiento**.

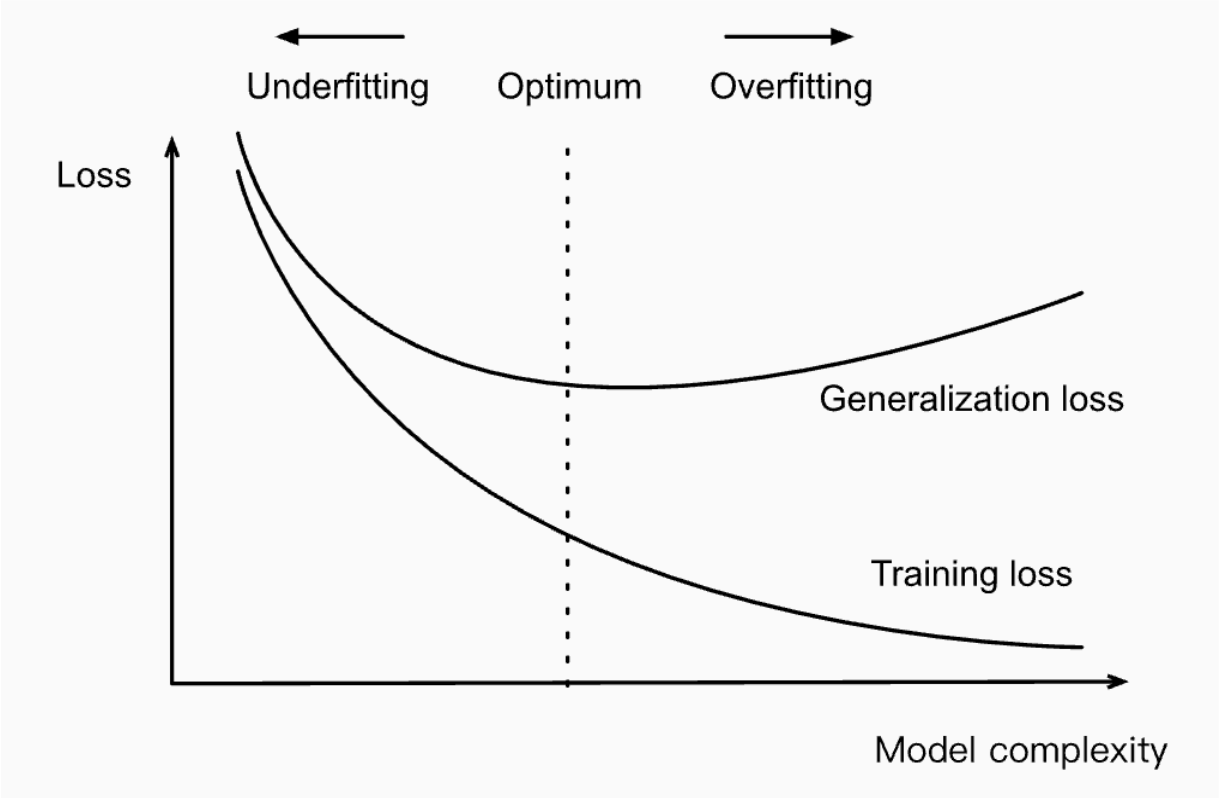
La diferencia entre ambos es pequeña, lo que indica un sobreajuste mínimo o nulo. Cabe destacar que **el rendimiento del conjunto de validación siempre será inferior al del conjunto de entrenamiento**.

NOTA:

- En modelos de redes neuronales la curva de entrenamiento suelen mostrar la tendencia: rendimiento en el entrenamiento generalmente comienza bajo y mejora a medida que aumenta el tamaño del conjunto de datos.

- Mientras que en modelos de clasificador árbol de decisión, se observa que, a medida que se añaden más datos, el rendimiento en los datos de entrenamiento puede disminuir ligeramente.

• OVERFITTING



1. Al comienzo del entrenamiento, la optimización y la generalización están correlacionadas: en cuanto menor sea la pérdida de datos de entrenamiento, menor será la pérdida de datos de prueba.

Mientras esto sucede, se dice que su modelo no es apto: todavía hay progreso por hacer; la red no ha modelado todos los patrones relevantes en los datos de entrenamiento.

2. Después de un cierto número de iteraciones en los datos de entrenamiento, la generalización deja de mejorar, las métricas de validación se estancan y luego comienzan a degradarse: el modelo está comenzando a sobreajustarse.

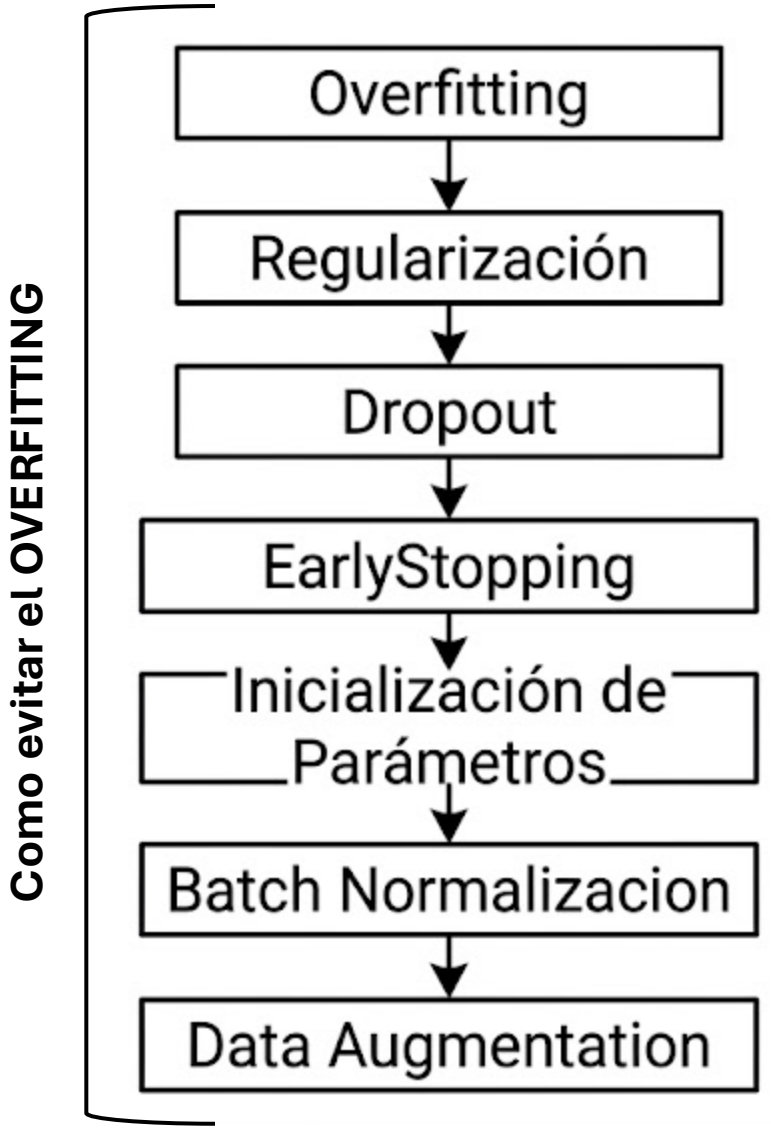
Es decir, está empezando a aprender patrones que son específicos de los datos de entrenamiento pero que son engañosos o irrelevantes cuando se trata de nuevos datos.

• OVERFITTING

Causas del sobreajuste:

- Datos de entrenamiento con ruido (datos no válidos)
- Características ambiguas (datos bien etiquetados pero ambiguos -> en clasificación el mismo dato en dos conjuntos)
- Características raras y correlaciones falsas.
- Muchas épocas de entrenamiento.

Un modelo no se evalúa por qué tan bien recuerda los datos de entrenamiento, sino por qué tan bien predice datos que nunca ha visto.





02

Regularización

Algunas Técnicas de Regularización son:

- Reducir el tamaño de la red

- Añadir “regularización a los pesos”.

- Manejar DROPOUT

entre otras …..

Las **Técnicas de Regularización** son un conjunto de mejores prácticas que impiden activamente la capacidad del modelo para ajustarse perfectamente a los datos de entrenamiento, con el objetivo de hacer que el modelo funcione mejor durante la validación.

A esto se le llama “regularizar” el modelo, porque tiende a hacer el modelo más simple, más “regular”, su curva más suave, más “genérica”; por lo tanto, es menos específico para el conjunto de entrenamiento y más capaz de generalizar acercándose más la variedad latente de los datos.

La Regularización permite reducir la diferencia entre el error de entrenamiento y el error de generalización.

L2 Regularization

Regularización por penalización de tamaño de parámetros: obligar a que los parámetros sean muy pequeños, interviniendo la función de error agregando una penalización al final.

Theta representa a todos los parámetros de la red : Θ = (W, b, …., U, c)

$$L = \frac{1}{n} \sum_{i=1}^{n} error(\hat{y}^{(i)} - y^{(i)}) \quad \Rightarrow \quad L' = \frac{1}{n} \left(\sum_{i=1}^{n} error(\hat{y}^{(i)} - y^{(i)}) + \beta * penalización(\Theta) \right)$$

Una de las funciones penalización es "L2 Regularization" (En Keras = weight decay): En cada paso del descenso del gradiente achica lo parámetros aplicando penalización logarítmica en base 0,1, 0,01, 0,001, etc.

Esta nueva función de error depende de los errores que está cometiendo la red con los ejemplos más una penalización sobre los parámetros. **Esto no cambia la capacidad de la red, sino que penaliza los parámetros grandes.**

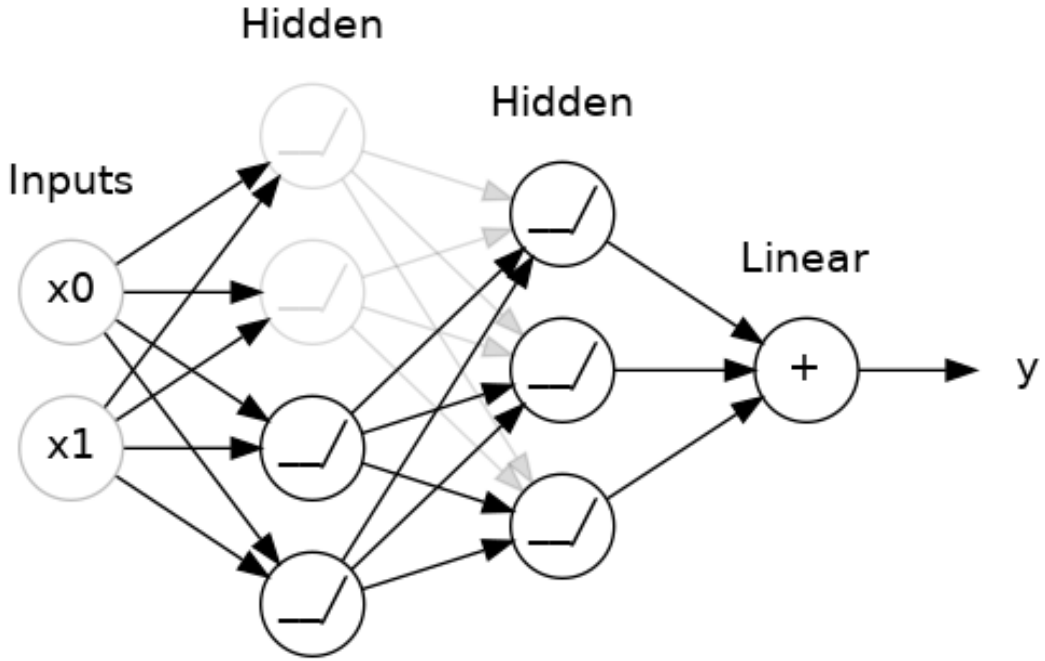
DropOut

Implementar DROPOUT es **“apagar” un porcentaje de neuronas (al azar) en las capas de la red antes de pasar un Batch.**

Dropout es SOLO durante el entrenamiento y por ende **sobre el dataset de entrenamiento**. Para el test no se usa.

Lo que estamos haciendo es hacer más complejo el aprendizaje a la red (la estresamos):

- Antes de pasar el batch, se apaga un % de neuronas al azar en alguna o todas las capas.
- Para el siguiente batch, “se encienden” todas las neuronas y se vuelven a apagar otras al azar.
- En general, en las últimas capas se tiende a usar % de apagado más bajo (para evitar perder características importantes).



DropOut

Demostración gráfica de cómo impacta el uso de dropout al error de test de una red neuronal.

Cinco data sets de imágenes (MNIST, SVHN, CIFAR-10, CIFAR-100, ImageNet).

Experimento realizado en el departamento de las ciencias de la computación de la Universidad de Toronto.

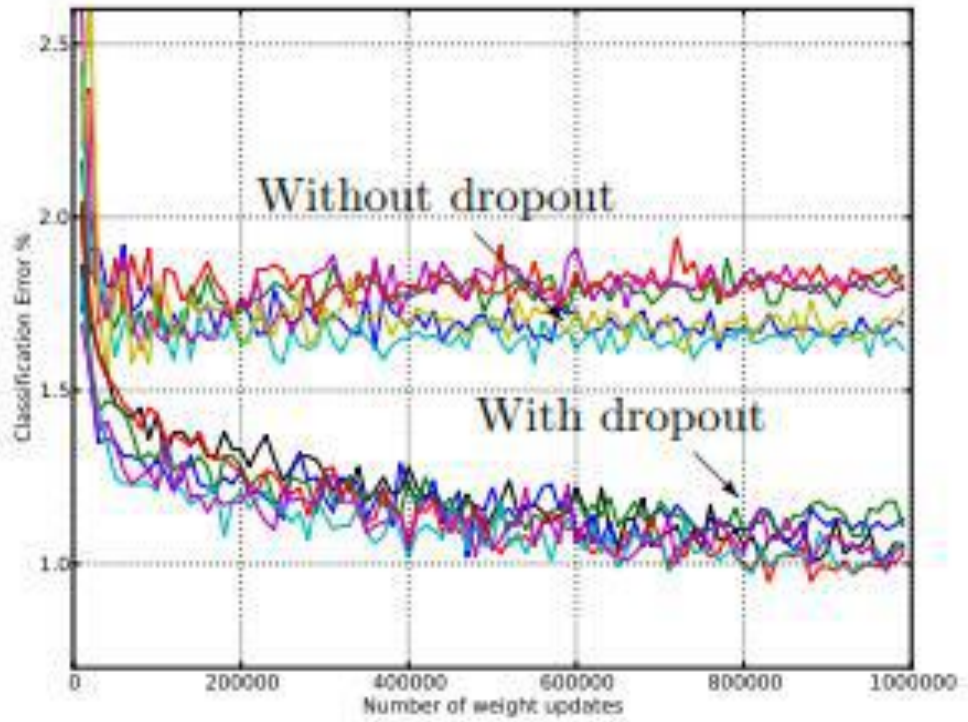
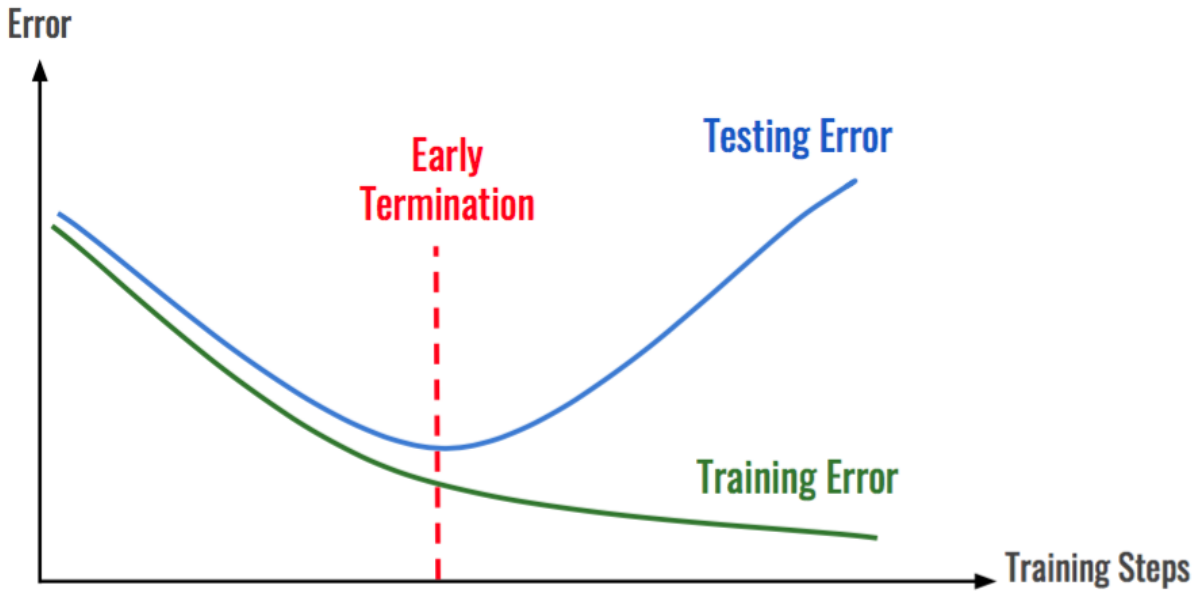


Figure 4: Test error for different architectures with and without dropout. The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

Early Stopping

Se trata de aplicar ciertas reglas para saber **cuándo es momento de parar el entrenamiento**, de forma que no se produzca sobreajuste a los datos de entrada, ni tampoco subajuste.

La regla más extendida sería la de entrenar el modelo monitorizando su rendimiento y guardando sus parámetros al finalizar cada época, hasta que apreciemos que el error de validación aumenta de forma sostenida. Así, nos quedaremos con el modelo que teníamos justo en el momento anterior.

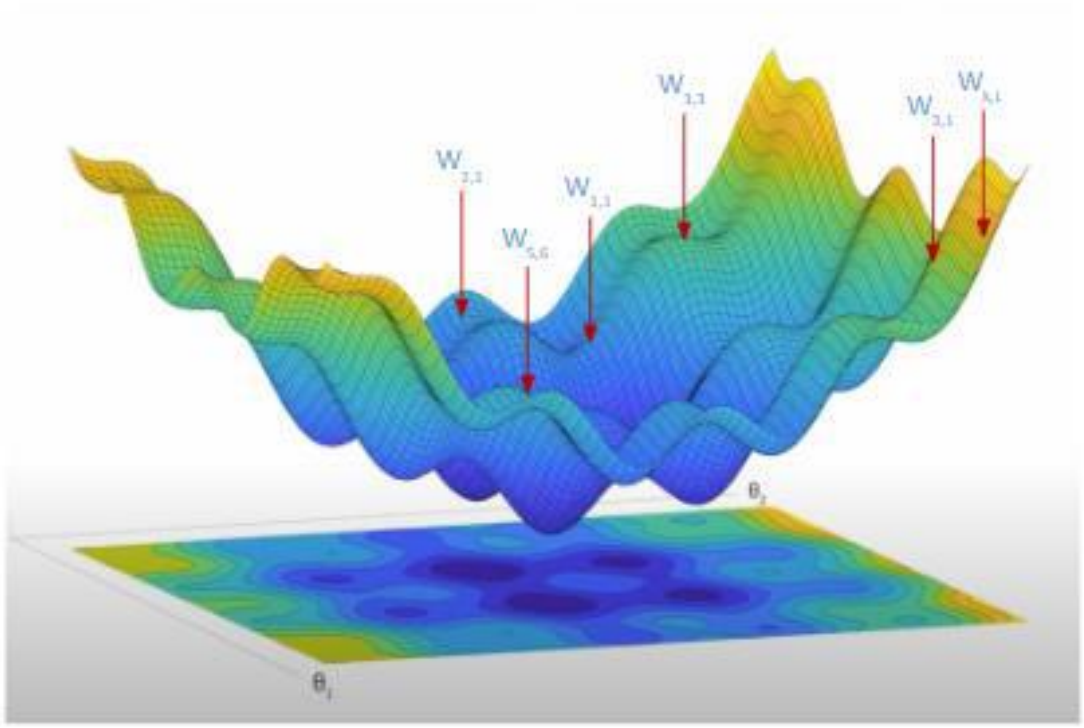


Inicializar Parámetros

Se entiende que **los parámetros (pesos y bias)** deben, en general, ser **números pequeños y en general rondar en torno al cero**.

Recordatorio: El descenso del gradiente es el encargado de actualizar los pesos una vez que comienza el entrenamiento.

Inicializar los pesos es indicar al algoritmo del descenso del gradiente **el punto de partida** para comenzar a descender hasta encontrar todos los parámetros que minimizan la función de error.



Inicializar Parámetros

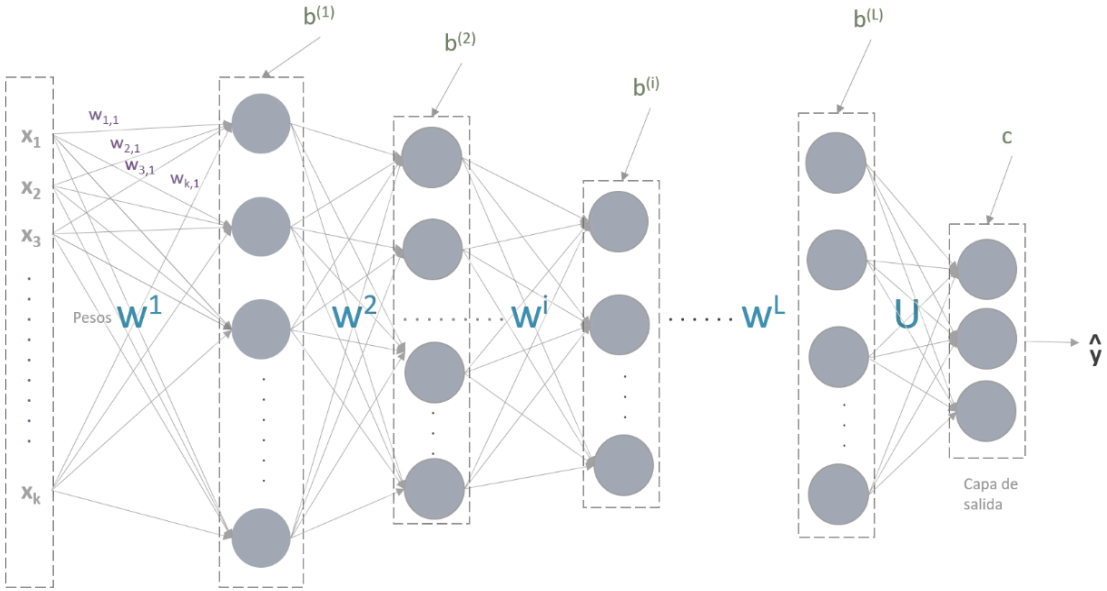
Existe un proceso que, al no ser ejecutado correctamente siguiendo las mejores prácticas, podría generar una arquitectura de red neuronal que finalmente no lograra “aprender” o que demorara muchísimo tiempo en su proceso.

Esta tarea se llama **inicialización de los parámetros**. Inicializar o no los bias “b” y “c”, o para ser más preciso, inicializarlos todos en cero, no impacta el correcto entrenamiento.

Inicializar los pesos “W”, “U” es una tarea crítica que debe abordarse con cuidado. Los pesos no pueden ser inicializados con ceros ya que el descenso del gradiente nunca comenzaría a bajar.

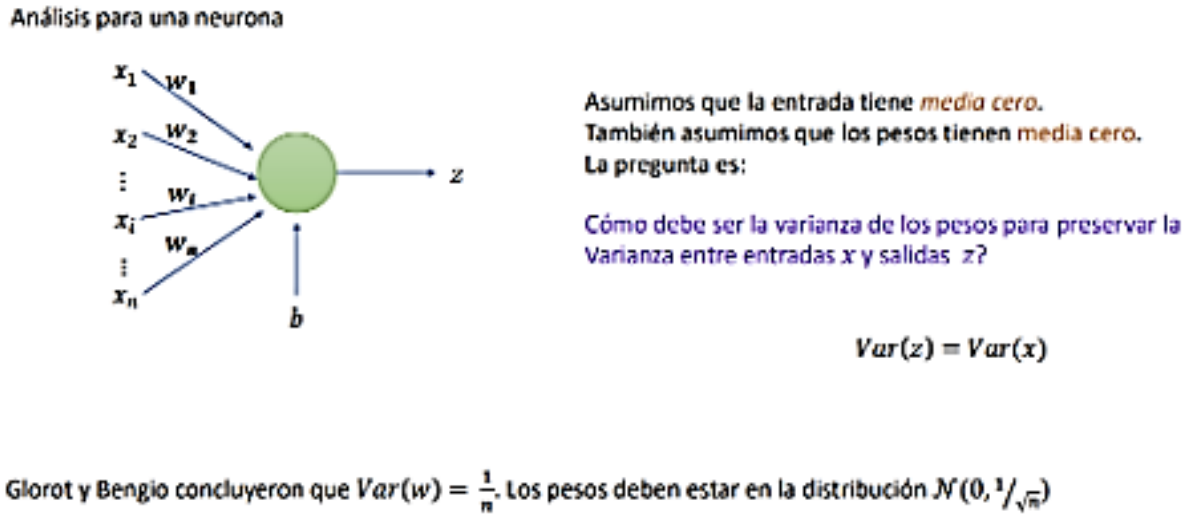
En general, no inicializar correctamente los pesos “W” y “U” podría causar alguno de los siguientes efectos:

- “Desvanecimiento del gradiente”
- “Explosión del gradiente”



Inicialización Xavier

$$w^i = \text{np.random.randn}(k_i, K_{i-1}) * \sqrt{\frac{1}{K_{i-1}}}$$



Esta inicialización se utiliza generalmente para Sigmoid o Tanh como función de activación.

Esta técnica, también llamada Glorot Initialization, donde inicializa los bias en cero y **los pesos según la siguiente fórmula** (esto asegura que la varianza de todos los pesos es 1), en la cual considera la cantidad de neuronas de la capa anterior (i-1) y la capa actual (i).

Genera números al azar entre el número de neuronas de la capa anterior y la cantidad de neuronas de la capa actual y los pondera por la raíz cuadrada de uno dividido por el número de neuronas de la capa anterior i menos 1.

Inicialización He-et-al

$$w^i = \text{np.random.randn}(k_i, K_{i-1}) * \sqrt{\frac{2}{K_{i-1}}}$$

Asumimos que los pesos tienen media cero. La pregunta es:

Cómo debe ser la varianza de los pesos para preservar la varianza entre las entradas x y la salida z?

$$Var(z) = Var(x)$$

He et al. concluyó que $Var(w) = \frac{2}{n}$. Los pesos tienen que estar en la distribución $\mathcal{N}(0, \sqrt{2/n})$

Esta inicialización se utiliza generalmente para Relu como función de activación (Utilizada en Keras).

Esta técnica, también llamada Glorot Initialization, donde inicializa los bias en cero y **los pesos según la siguiente fórmula**, en la cual considera la cantidad de neuronas de la capa anterior (i-1) y la capa actual (i).

Genera números al azar con distribución normal entre el número de neuronas de la capa anterior y la cantidad de neuronas de la capa actual y los pondera por la raíz cuadrada de dos dividido por el número de neuronas de la capa anterior i menos 1.

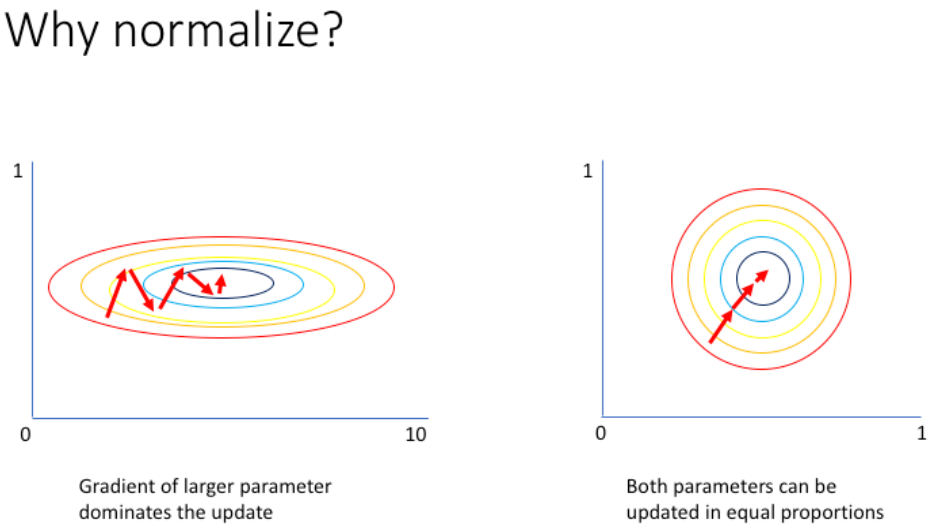
Escalar valores de entrada

Otra buena práctica es inicializar o normalizar los valores de entrada (X) para **evitar que alguna dimensión sea interpretada como más o menos importante que la otra.**

Normalizando las entradas (X), en otras palabras, que **todas las dimensiones estén en la misma escala**, permite que la función de costo es **minimizada de manera más rápida y directa**, lo que se traduce en un entrenamiento más efectivo y eficiente.

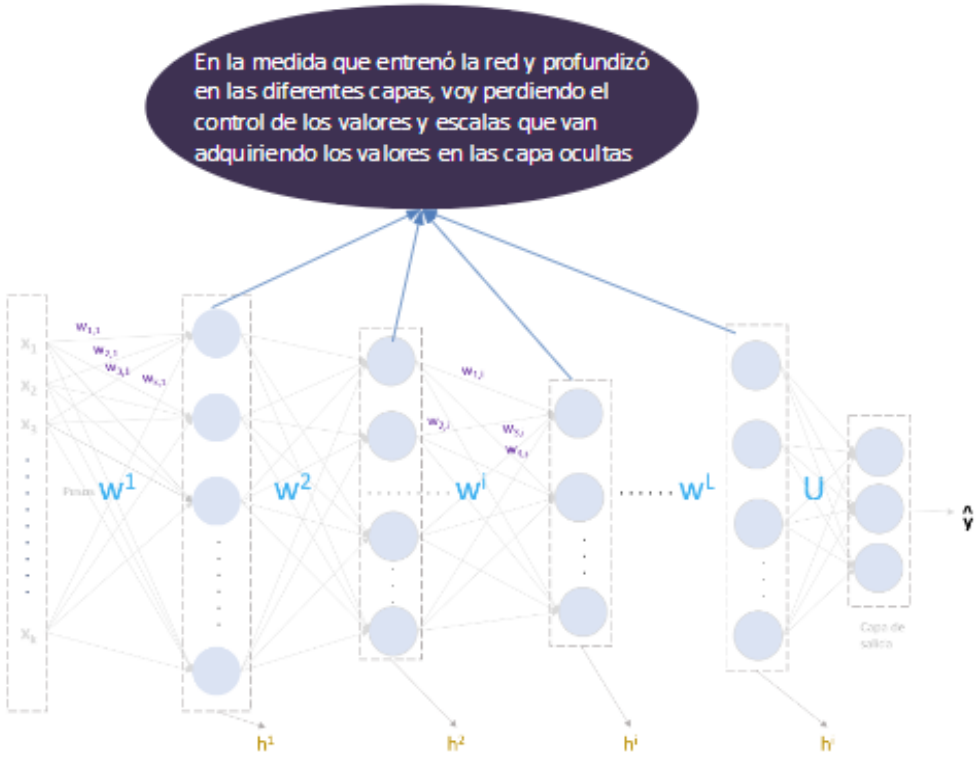
En general, lo que se hace para escalar las entradas es forzar todo a tener una media de 0 y varianza 1.

Otra manera es todo a estar entre 0 y 1, como se hace cuando se trata de imágenes, donde se divide el valor de cada pixel por 255.



Sin normalización de los inputs la optimización puede demorar en converger al punto mínimo (óptimo).

Batch Normalization



BATCH NORMALIZATION consiste en **forzar los valores de los pesos de cada capa** al rango que necesitamos para que la red siga entrenando de buena manera, esto hace que el tiempo de duración del entrenamiento sea más rápido, también ayuda de cierta manera a regularizar la red y a reducir el error de generalización.

BATCH NORMALIZATION consiste básicamente en añadir un paso extra, habitualmente entre las neuronas y la función de activación, con **la idea de normalizar las activaciones de salida.**

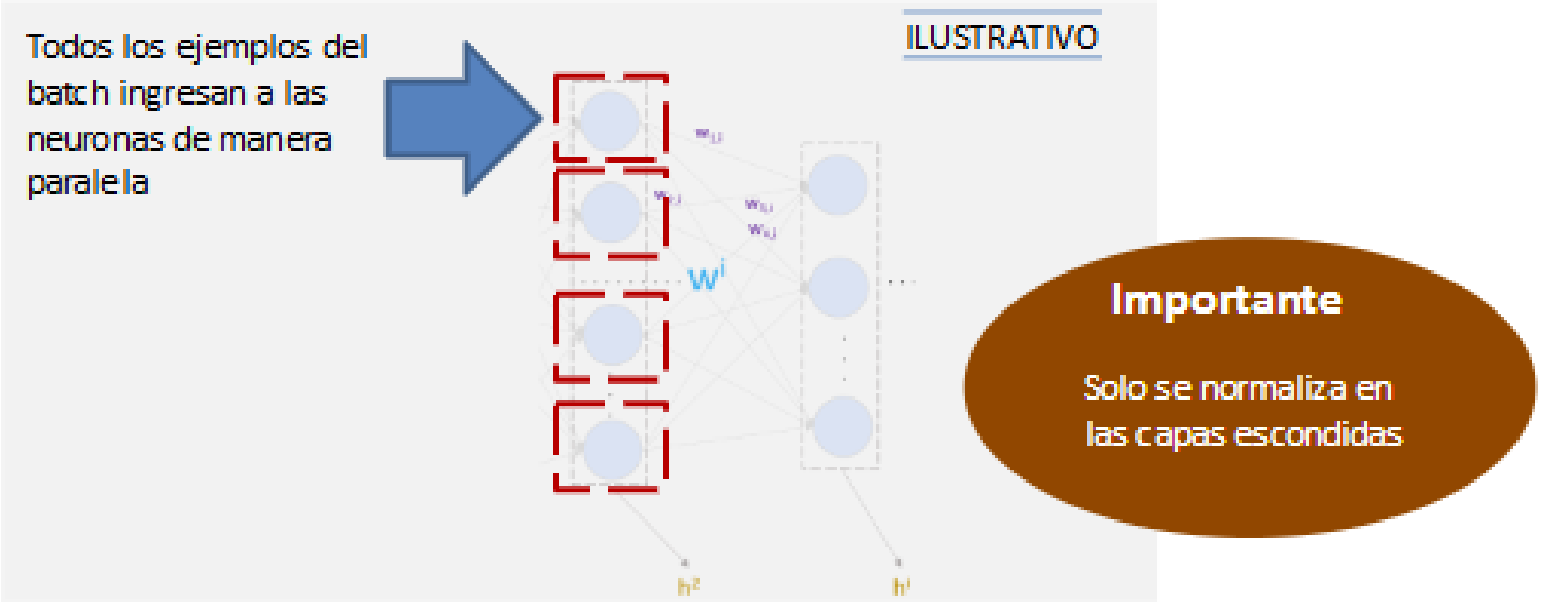
Lo ideal es que la normalización se hiciera usando la media y la varianza de todo el conjunto de entrenamiento, pero si estamos aplicando el descenso del gradiente estocástico para entrenar la red, se usará la media y la varianza de cada mini-lote de entrada.

Nota: cada salida de cada neurona se normalizará de forma independiente, lo que quiere decir que en cada iteración se calculará la media y la varianza de cada salida para el minilote en curso.

Batch Normalization

Los valores idealmente se deben inicializar en valores pequeños, por ejemplo**: media 0 y varianza 1; rangos entre -1 y 1 o rangos entre 0 y 1, etc.**

El prefijo “batch” se refiere a que la normalización de una dimensión de los datos, se realiza sobre todos los ejemplos del batch que en ese momento se está evaluando en la capa oculta h sub i.

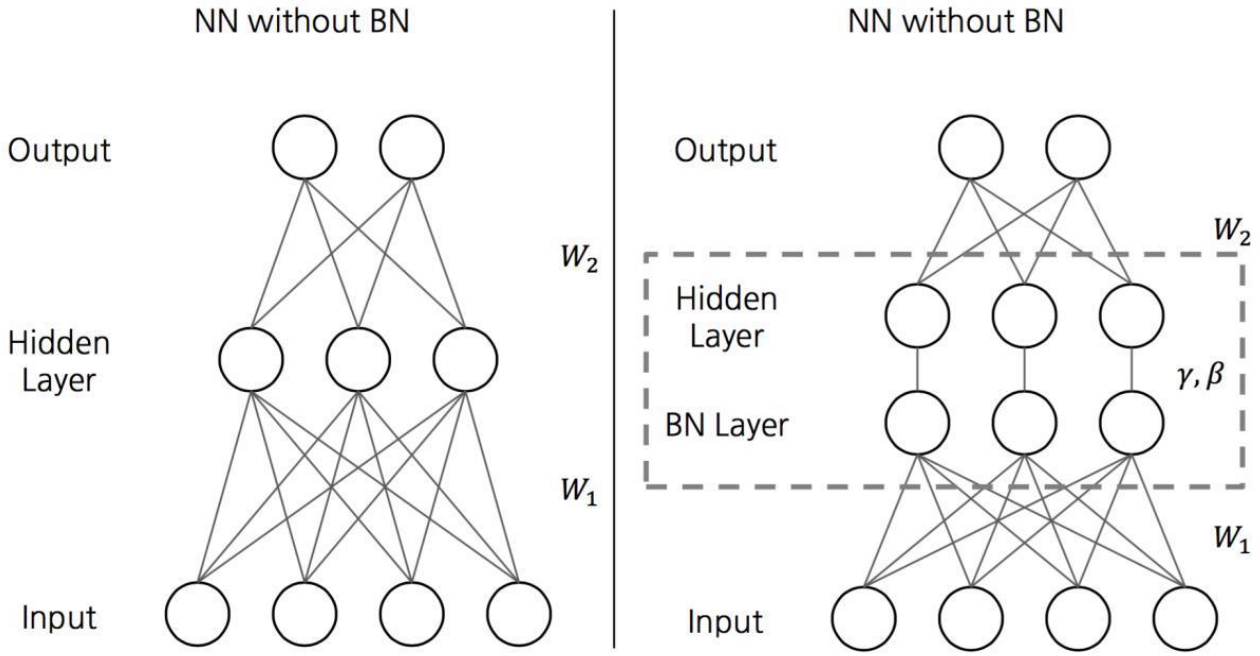


Batch Normalization

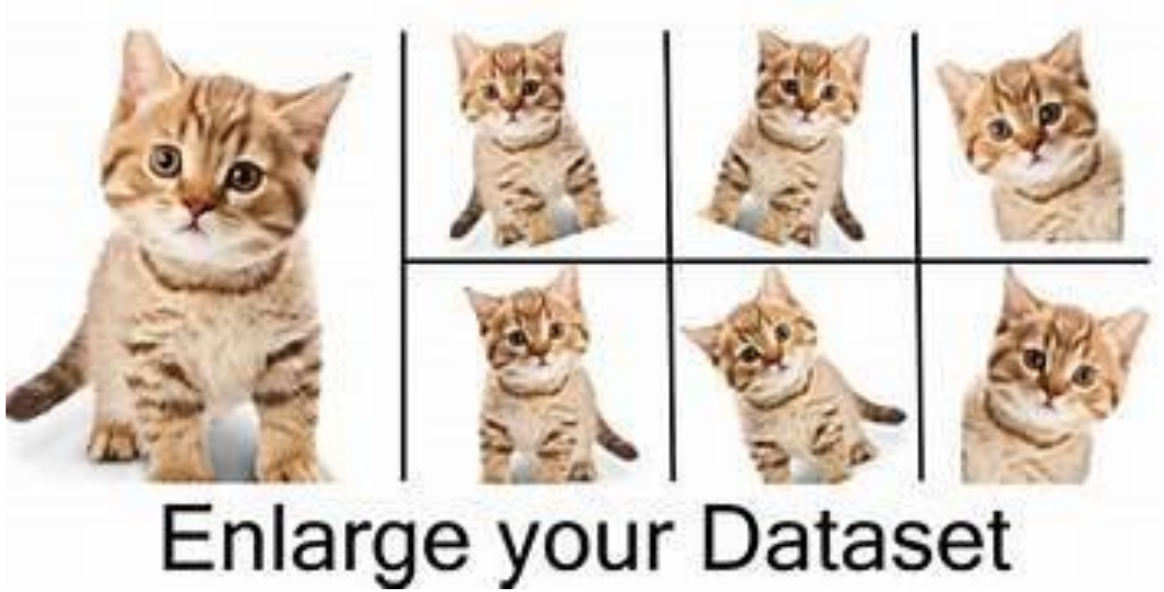
Así, BATCH NORMALIZATION agrega una nueva capa, después de la capa oculta h supra i.

La normalización por lotes afecta el resultado de la capa de activación anterior, **al restar la media del batch y luego dividirla por la desviación estándar del batch.**

El resultado final es que la normalización por lotes agrega dos parámetros entrenables adicionales a una capa: **la salida normalizada que se multiplica por un parámetro gamma (desviación estándar) y el parámetro beta adicional (media).**



Data Augmentation



Ejemplos de transformaciones son:

- Voltear la imagen en horizontal / vertical
- Rotar la imagen X grados.
- Recortar, añadir relleno, redimensionar, ...
- Aplicar deformaciones de perspectiva
- Ajustar brillo, contraste, saturación, ...
- Introducir ruido, defectos, ...
- Combinaciones de las anteriores

Una de las **cosas más caras** en Deep Learning es conseguir más **datos etiquetados**.

Siempre traer más datos es lo mejor, pero muy caro.

DATA AUMENTATION es la técnica de que a partir de los mismos datos se generan más datos.

Ejemplo: si tengo una foto, se podría rotar o cortar en algún lugar (“random crop”).

Lo veremos con más detalle en la siguiente clase...

Duoc UC®

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS ACREDITADO

Comisión Nacional de Acreditación CNA-Chile

DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.